

Conceptos y arquitectura del asistente

Un asistente es un programa que gestiona conversaciones con un LLM, no un único texto enviado una vez.

Este documento cubre: qué es un asistente, cómo se organiza en módulos, qué es el **estado**, la **configuración** y la **personalización** por perfiles.

Objetivos

- Diferenciar prompt aislado vs asistente estructurado.
- Describir una arquitectura mínima en Python.
- Modelar `user_state` y `assistant_config` como estructuras separadas.
- Ver cómo los perfiles cambian el comportamiento sin cambiar el modelo.

1) Qué es un asistente conversacional

Un **asistente** es una capa de software que:

1. Recibe input del usuario (texto, parámetros).
2. Lee y actualiza **estado** de la sesión.
3. Aplica **configuración** del producto (tono, rol, límites).
4. **Construye** el prompt del turno (instrucciones + contexto + mensaje).
5. Llama al LLM.
6. Devuelve la respuesta y **persiste** lo necesario para el siguiente turno.

El modelo **no recuerda** entre llamadas. El asistente **sí** puede recordar si tu código reenvía estado e historial.

Prompt aislado vs asistente

```
# A) Prompt aislado – una sola llamada, sin sistema alrededor
client.models.generate_content(
    model="gemini-3-flash-preview",
    contents="Explícame qué es una API REST.",
)

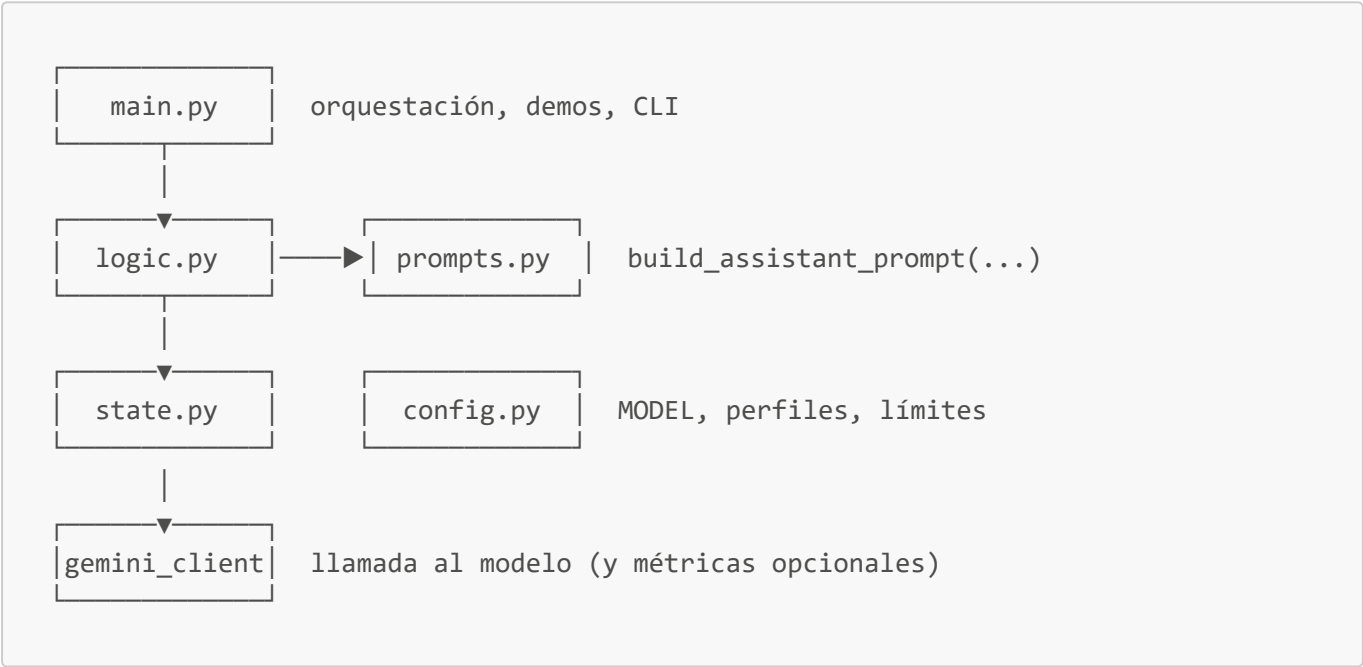
# B) Asistente – el contenido lo construye tu capa de aplicación
prompt = build_assistant_prompt(
    assistant_config=config,
    user_state=state,
    user_message="Explícame qué es una API REST.",
)
client.models.generate_content(model=config["model"], contents=prompt)
```

Aspecto	Prompt aislado	Asistente
Turnos	1	N
Memoria	No	Estado + historial (opcional)
Personalización	Manual cada vez	Perfiles / config
Mantenimiento	Bajo si es demo única	Mejor si el producto crece

La **Demo 1** ([01_de_prompt_suelto_a_asistente_ejemplos.ipynb](#)) compara ambos enfoques con la misma pregunta y un segundo turno ("¿cómo me llamo?") para mostrar el límite del enfoque sin arquitectura.

2) Arquitectura básica

Patrón recomendado:



Separación de responsabilidades

Módulo	Responsabilidad	No debería
<code>config.py</code>	Constantes, perfiles por defecto, límites	Llamar a Gemini
<code>state.py</code>	Crear/leer/actualizar estado e historial	Construir prompts largos
<code>prompts.py</code>	Plantillas y <code>build_*_prompt</code>	Validar negocio complejo
<code>logic.py</code>	Un turno: validar → prompt → llamar → guardar	<code>print</code> de marketing
<code>main.py</code>	Escenas de demo, entrada/salida usuario	Lógica de negocio pesada

Regla práctica: si cambias el **tono** del asistente, tocas `config` o `prompts`; si cambias **qué se recuerda**, tocas `state`; si cambia el **flujo del turno**, tocas `logic`.

3) Estado (**user_state**)

El **estado** es lo que la sesión **acumula** sobre el usuario y la conversación. Vive en memoria en estos ejemplos (sin base de datos).

Ejemplo mínimo:

```
def crear_estado_inicial() -> dict:
    return {
        "user_profile": {
            "nombre": "",
            "nivel": "junior",
            "tema_actual": "",
        },
        "messages": [],          # historial [{role, content}, ...]
        "summary": "",          # opcional: resumen comprimido (Sprint 5 U3)
        "turnos": 0,
    }
```

Funciones típicas en **state.py**:

```
def append_user(state: dict, texto: str) -> None:
    state["messages"].append({"role": "user", "content": texto.strip()})
    state["turnos"] += 1

def append_assistant(state: dict, texto: str) -> None:
    state["messages"].append({"role": "assistant", "content": texto.strip()})

def ultimos_n(state: dict, n: int) -> list:
    return state["messages"][-n:]
```

Qué va al estado

- Nombre, preferencias declaradas por el usuario.
- Historial de mensajes (o ventana + resumen).
- Metadatos de sesión (contador de turnos, último tema).

Qué no va al estado

- API keys.
- Instrucciones fijas del producto (eso es **config**).
- Textos de plantilla (eso es **prompts**).

4) Configuración (**assistant_config**)

La **configuración** describe **cómo debe comportarse el asistente** como producto, independientemente de un usuario concreto (salvo que elijas personalizar por perfil).

Aqui se configuran valores como el modelo, la temperatura, idioma, etc... que se usarán en el asistente.

```
ASSISTANT_CONFIG_DEFAULT = {
    "model": "gemini-3-flash-preview",
    "temperature": 0.3,
    "perfil_activo": "mentor",
    "max_turnos_historial": 6,
    "idioma_respuesta": "español",
    "max_palabras": 200,
}
```

Puedes cargar perfiles completos desde `config.py`:

```
PERFILES = {
    "junior": {
        "rol": "Eres un compañero de estudio amable. Explicas con ejemplos cortos.",
        "nivel_explicacion": "básico",
    },
    "senior": {
        "rol": "Eres un ingeniero senior. Vas al grano, asumes conocimientos previos.",
        "nivel_explicacion": "avanzado",
    },
    "mentor": {
        "rol": "Eres un mentor pedagógico. Guías con preguntas y pasos.",
        "nivel_explicacion": "intermedio",
    },
}
```

Estado vs configuración

Pregunta	¿Estado o config?
¿Cómo se llama el usuario?	Estado (<code>user_profile</code>)
¿Qué perfil usa el producto por defecto?	Config (<code>perfil_activo</code>)
¿Qué preguntó hace tres turnos?	Estado (<code>messages</code> / <code>summary</code>)
¿Temperatura del modelo?	Config
¿Tema que estudia hoy Ana?	Estado

Mezclarlos en un solo `dict` sin criterio complica los tests y seguridad del sistema.

5) Personalización

Personalizar no es “cambiar el modelo”. Es cambiar **instrucciones y parámetros** que entran en el prompt según perfil o usuario.

Flujo típico:

```
def resolver_perfil(config: dict) -> dict:
    clave = config["perfil_activo"]
    if clave not in PERFILES:
        raise ValueError(f"Perfil desconocido: {clave}")
    return PERFILES[clave]

def build_assistant_prompt(config: dict, state: dict, user_message: str) -> str:
    perfil = resolver_perfil(config)
    nombre = state["user_profile"].get("nombre") or "estudiante"
    return f"""
{perfil["rol"]}

Configuración de sesión:
- Usuario: {nombre}
- Nivel declarado: {state["user_profile"].get("nivel", "junior")}
- Idioma: {config["idioma_respuesta"]}
- Máximo aproximado: {config["max_palabras"]} palabras

Mensaje del usuario:
{user_message.strip()}
""".strip()
```

La **Demo 2** ([02_estado_y_configuracion_ejemplos.ipynb](#)) fija la misma pregunta y alterna perfiles **junior**, **senior** y **mentor** para comparar tono y profundidad.

Buenas prácticas

- Perfiles en **un diccionario** o fichero JSON, no dispersos en `if/else`.
- El usuario puede **sobrescribir** nivel en `user_profile`; el perfil del producto define el estilo base.
- Imprime el prompt en desarrollo antes de gastar tokens (herencia Sprint 5).

6) Errores frecuentes

1. **Un solo prompt global** que crece en cada turno sin `state.py`.
2. **Guardar el historial solo en variables sueltas** en `main.py`.
3. **Perfil hardcoded** en cada llamada en lugar de `assistant_config`.
4. **Duplicar el mensaje del usuario** en historial y en `user_message` del prompt (mismo problema que en Context Engineering — construir el prompt con criterio).
5. **Asumir que el asistente “entiende” la sesión** sin que Python reenvíe datos.

Resumen

- Un **asistente** orquesta turnos; el LLM solo genera texto para uno de ellos.

- **Arquitectura** = módulos con responsabilidad clara.
- **Estado** = lo que cambia por usuario y sesión; **config** = cómo se comporta el producto.
- **Personalización** = perfiles y parámetros que moldean el prompt sin cambiar de modelo.